

PRBot - Exploring AI Summarization Outputs for Issues, PRs and Repos

Don Syme – my opinions but based on work of all of GitHub Next

AI Outputs Overview

- **Summarization**
- Chat
- Generative
- Critique
- Indexing

AI Outputs Overview

- **Summarization**
 - Keywords
 - Title
 - Summary
 - Abstract
 - Walkthrough
- Chat
- Generative
- Critique
- Indexing

AI Summarization – Example – PR Summary

`./prbot describe https://github.com/githubnext/prbot/pull/565`

▼ Summary

This pull request enhances the bot's ability to describe code changes, updates the test scores for the describe task, and simplifies the poetry generation code. It modifies `src/bot/bot-describe.ts` to improve the description quality and readability, updates `test/scores/boosted/describe-score.md` to report the latest results, and removes an unused case from `src/engine/poems.ts` to improve code clarity.

AI Summarization – Example – PR Walkthrough

`./prbot describe https://github.com/githubnext/prbot/pull/565`

▼ Code Walkthrough

- Simplify and improve the `filterForHallucinatedLocators` function to use a regular expression for matching lines with valid locators ([link](#))
- Add a comment to explain the `generateChangeByChangeDescriptionMarkdown` function, which creates a markdown string for each change ([link](#))
- Remove an outdated comment and a redundant line from the `addChangeByChangeLines` function, which adds lines to the change-by-change markdown ([link](#),[link](#))
- Remove the **unused case** for "greekepic" from the `expectedLines` function in the `OverallPoetryStrategy` class, which determines the number of lines for a poetry kind ([link](#))
- Update the overview, comparison, summaries, walkthrough, and JSON sections in the `describe-score.md` file, which reports the performance and progress of the describe task ([link](#),[link](#),[link](#),[link](#),[link](#))

Note: Grouping is implemented here, no titles for groups (no “sections”)

Note: Small Inaccuracies are common. May be possible to remove ~90% of these over time but some will remain.

AI Summarization – Example – PR Poem

`./prbot describe https://github.com/githubnext/prbot/pull/565`

*The docs of the change prediction engine
Were scattered and lacked some cohesion
So they moved them to README
And renamed some endpoints neatly
And added some options for Python*

Note: humorous but seems well-received, good for team cohesion

AI Summarization – Example – Repo Summary

```
./prbot summarize https://github.com/githubnext/HeyGitHub --include "src/**/*.ts"
```

▼ Summary

The repository `githubnext/HeyGitHub` contains the source code for a VS Code extension that allows users to interact with GitHub repositories using voice and text commands. It uses speech recognition and synthesis, natural language processing, and OpenAI API to provide features such as code generation, editing, explanation, and navigation. It also includes some utilities and tests for the extension.

AI Summarization – Example – Repo Walkthrough

`./prbot summarize https://github.com/githubnext/HeyGitHub --include "src/**/*.ts"`

▼ Code Walkthrough

The repository is a VSCode extension that allows users to interact with GitHub and GitHub Copilot using voice commands. It uses an embedded speech server, an intent detection module, and an OpenAI API to process user utterances and execute various commands or queries.

The repository structure is as follows:

- The `src` directory contains the main source code for the extension, organized into subdirectories based on functionality.
 - The `audio_helper` directory contains files and snippets for interacting with the embedded speech server and processing user utterances. ([link](#),[link](#),[link](#),[link](#))
 - The `auth` directory contains files and snippets for authenticating against the GitHub Next waitlist and showing a warning message if the user does not have access to the "Hey, GitHub!" project. ([link](#))
 - The `chat` directory contains files and snippets for managing the chat interface between the user and the GitHub agent. It handles the messages from the webview, processes the user messages by invoking the intent detection and action mapping modules, and updates the webview with the agent responses and suggestions. ([link](#),[link](#),[link](#),[link](#))
 - The `ide` directory contains files and snippets for interacting with the VSCode editor and providing inline suggestions for code generation or code update commands. It also defines some types and utilities for the editor window and the document symbols. ([link](#),[link](#),[link](#),[link](#),[link](#))

The Principles

- **Utility**
- **Quality**
- **Handling Change**
- **Trust**
- **User Promotion**

The Principles

- **Utility**
 - The AI outputs must make some human task faster, or better
 - All new outputs take user attention. They aren't free.
- **Quality**
 - The AI makes mistakes, the user must either **approve/reject** or **tolerate** or **not care**
 - Users seem more tolerant of mistakes in **ephemeral** outputs
 - The longer the AI outputs, the **greater the chance of mistakes**
- **Change**
 - AI outputs should be **fresh** and **up-to-date**
 - **User-editing of outputs** is often requested, but beware it's a quagmire in a changing context
- **Trust**
 - **Authorship** must be understood - the user must understand that they are reading AI outputs
 - A human must **explicitly** take **ownership** after **checking**
- **User Promotion**
 - AI outputs should be labelled until explicitly **promoted (= integrated(edited) or accepted or ticked or reported)** by user.
 - In editing contexts this is best done via **ghost suggestions**, in non-editing contexts this is harder
 - There is a **spectrum of promotion UX possible**. **User editing** of outputs is at one end. **Mark as misleading** is at the other.
 - The **longer** and more **permanent** the AI outputs, the **more chance** the user will want to edit them

AI Summarization - Techniques

Techniques:

- Process all inputs, chunk by chunk, snippet by snippet, file by file etc.
- Combine summaries hierarchically
- Divide-and-conquer outputs to stay within prompt limits
- Attempt to give appropriate context
- Operate within a budget
- Make it all incremental

AI Summarization – Example – PR Walkthrough

`./prbot describe https://github.com/githubnext/prbot/pull/565`

▼ Code Walkthrough

- Simplify and improve the `filterForHallucinatedLocators` function to use a regular expression for matching lines with valid locators ([link](#))
- Add a comment to explain the `generateChangeByChangeDescriptionMarkdown` function, which creates a markdown string for each change ([link](#))
- Remove an outdated comment and a redundant line from the `addChangeByChangeLines` function, which adds lines to the change-by-change markdown ([link](#),[link](#))
- Remove the **unused case** for "greekepic" from the `expectedLines` function in the `OverallPoetryStrategy` class, which determines the number of lines for a poetry kind ([link](#))
- Update the overview, comparison, summaries, walkthrough, and JSON sections in the `describe-score.md` file, which reports the performance and progress of the describe task ([link](#),[link](#),[link](#),[link](#),[link](#))

Note: Grouping is implemented here, no titles for groups (no “sections”)

Note: Small Inaccuracies are common. May be possible to remove ~90% of these over time but some will remain.

AI Summarization Latencies

Using latest models

Any PR/Issue discussion summarization:

- ~15sec?

Any PR Summarization:

- ~2 Minutes?

Any Repo Summarization:

- 10+ Minutes if from scratch?

Testing (PR summarization)

- 200 PRs, fairly randomly chosen, range of sizes
- Capture baselines, allow precise reply
- Score changes
 - Flesch
 - ROUGE (precision, recall), BLEU v. Human summary in PR
 - Adhoc statistics (length, grouping)
- **Eyeball** stats and differences to **shape** outputs based on **user feedback**

Some Lessons

- Quality is critical – users notice all mistakes
- Utility is subtle – many different participants in a PR
- Size matters – large AI outputs are hard
- Change matters – freshness is critical, but users also want control
- Trust matters – label all outputs!
- Shaping the outputs reliably is hard but possible
- Successful UX is hard
 - Ghost text is **amazing: small, non-intrusive, generative** AI outputs in a **changing context** with never-ending **user review**
 - Immutable “AI Opinion” seems plausible for **small** summarization outputs
 - Struggling with **large** summarization outputs, with too many mistakes